Activation Functions

Activation Functions

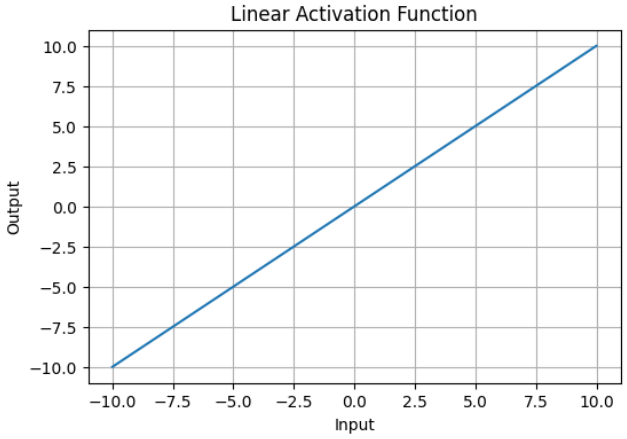
- It is a **mathematical function** applied to the output of a neuron.
- It introduces **non-linearity** into the model, allowing the network to learn and represent complex patterns in the data.
- Without this non-linearity feature a neural network would behave like a linear regression model no matter how many layers it has.
- Activation function decides whether a neuron should be **activated** by calculating the **weighted sum of inputs and adding a bias term**.
- This helps the model make **complex decisions** and predictions by introducing **non-linearities to the output of each neuron**.

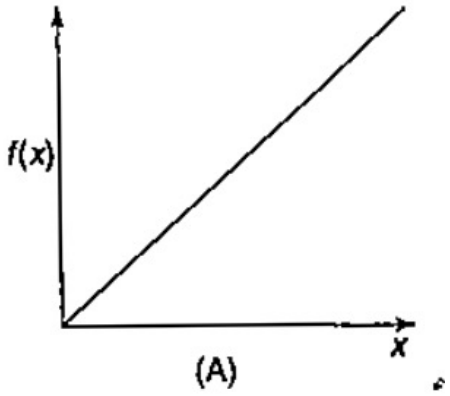
The most commonly used activation function are listed below:

1. Identity(Linear) Function
2. Binary Step Function(Threshold function)
3. Bipolar Step Function
4. Sigmoidal Function
    1. Binary Sigmoid Function
    2. Bipolar Sigmoid Function
5. Hyperbolic Tangent Function
6. ReLU (Rectified Linear Unit) Function

Identity(Linear) Activation Function

1. **Identity(Linear) Function:** It is a linear function resembles straight line define: f(x)=x or y=x for all x
   - No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.
   - The range of the output spans from (−∞ to +∞)(−∞ to +∞).
   - Linear activation function is used at just one place i.e. output layer.
   - Using linear activation across all layers makes the network's ability to learn complex patterns limited.



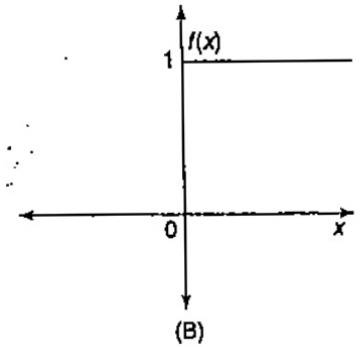


Binary Step Function Activation Function

2. Binary Step Function(Threshold): It is a threshold function resembles binary values define:

$$f(x) = \begin{cases} 1 & \text{if } x \geqslant \theta \\ 0 & \text{if } x < \theta \end{cases}$$

- Where θ represent the threshold value.
- This is widely used in single layer NN(perceptron's) to convert the net input to an output that is binary(0 or 1).
- It is used in Basic classification tasks like logic gate simulation(AND, OR, NOT).
- It looks like a linear functionality but discontinuity at threshold that x=0, the function jumps from 0 to 1, where linear functions cant have jumps; they are smooth lines.
- Here output is not proportional to input i.e. no matter how large the value of x output y is always flat.



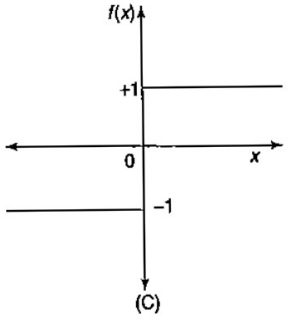
(B)

Bipolar Step Activation Function

3. Bipolar Step Function: It is also a threshold function resembles bipolar values, defined as:

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

- Where θ represent the threshold va
- This is widely used in single layer NN(bipolar logic system, Hopfield networks, Associative memory, etc) to convert the net input to an output that is bipolar(-1 or +1).
- It gives **symmetric decision boundary** around zero.
- It looks like a linear functionality but discontinuity at threshold that x=0, the function jumps from -1 to 1, where linear functions cant have jumps; they are smooth lines.
- Here output is not proportional to input i.e. no matter how large the value of x output y is always flat.



Sigmoid Activation Functions

4. Sigmoidal Function: The sigmoidal functions are widely used in **back-propagation nets** because of the relationship between the **value of the functions** at a point and the **value of the derivative at that point** which reduces the **computational burden** during training.

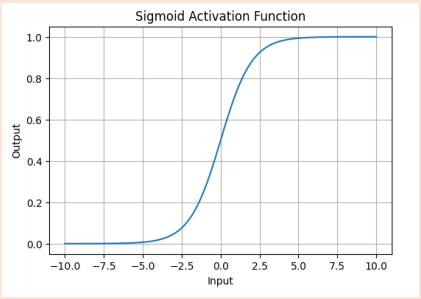
Sigmoidal functions are of two types

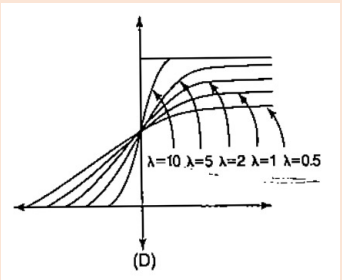
(i)Binary Sigmoid **(ii)Bipolar Sigmoid**

(i)Binary Sigmoid: It is also termed as **logistic sigmoid** function or **unipolar sigmoid** function. It can be defined as:

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

- Where λ represent the steepness parameter.
- This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.
- The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.

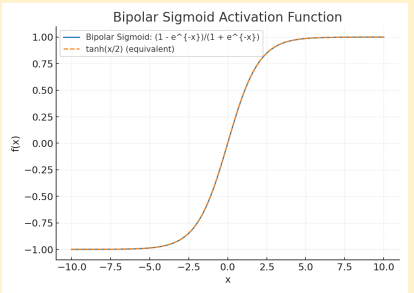




(ii)Bipolar Sigmoid(Tanh) : It is a hyperbolic tangent function and is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

- Where λ represent the steepness parameter.
- This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between -1 and 1..
- Value Range: Outputs values from -1 to +1.
- Non-linear: Enables modeling of complex data patterns.
- Use in Hidden Layers: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



Tanh Activation Function

- **(ii)Hyperbolic Tangent (Tanh) :** It is a hyperbolic tangent function and is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1.$$

Alternatively, it can be expressed using the sigmoid function:

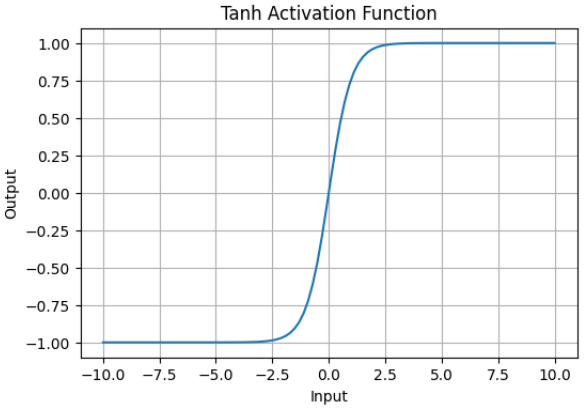
$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

$$f'(x) = \frac{\lambda}{2}[1 + f(x)][1 - f(x)]$$

The bipolar sigmoidal function is closely related to hyperbolic tangent function, which is written as

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- Where λ represent the steepness parameter.
- This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.
- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between -1 and 1..
- Value Range: Outputs values from -1 to +1.
- Non-linear: Enables modeling of complex data patterns.
- Use in Hidden Layers: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



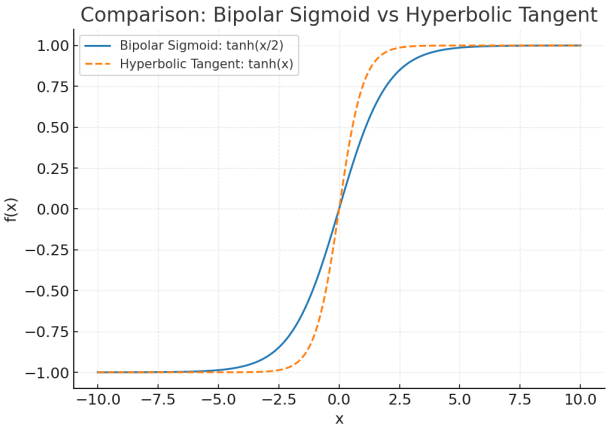
Bipolar Sigmoid:

- f(x)=tanh(x/2) → grows more slowly, smoother transition.
- It rises more gradually → better for smooth learning.

Hyperbolic Tangent:

- f(x)=tanh(x) → steeper transition around zero.
- **Tanh** rises more steeply → better when quick activation around zero is needed.

In practice: tanh is more commonly used in modern deep learning, but bipolar sigmoid is still valuable for specific cases where a gentler non-linearity improves stability.



ReLU Activation Function

4. Rectified Linear Unit(ReLU) : It is defined by A(x)=max(0,x) this means that if the input x is positive, ReLU returns x, if the input is negative, it returns 0.

That means:

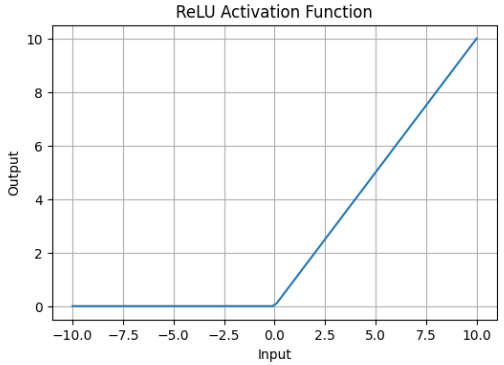
If x>0, then f(x)=x.

If x≤0x, then f(x)=0.

- **Value Range**: **Input range**: (−∞,+∞) and Output range [0,∞), meaning the function only outputs non-negative values.
- **Nature**: It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.

Advantage over other Activation:

- ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations.
- At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.
- Simple and computationally efficient.
- Helps reduce vanishing gradient problem (compared to sigmoid/tanh).
- Sparse activation (only positive neurons are active, improving efficiency).
- Works very well in deep networks and CNNs.



Why Are Activation Functions Necessary?

Neural networks consist of multiple layers where neurons process input signals and pass them to subsequent layers.

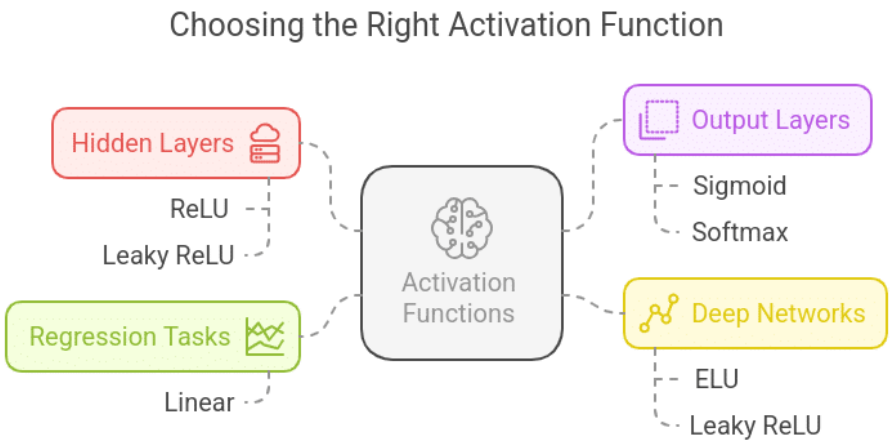
Everything inside a neural network becomes a basic linear transformation when activation functions are removed, which renders the network unable to discover complex features.

Key reasons why activation functions are necessary:

- **Introduce non-linearity**: Real-world problems often involve complex, non-linear relationships. Activation functions enable neural networks to model these relationships.
- **Enable hierarchical feature learning**: Deep networks extract multiple levels of features from raw data, making them more powerful for **Pattern Recognition**.
- **Prevent network collapse**: Without activation functions, every layer would perform just a weighted sum, reducing the depth of the network into a single linear model.
- **Improve convergence during training**: Certain activation functions help improve gradient flow, ensuring faster and more stable learning.

How to Choose the Right Activation Function?

- **For hidden layers,** ReLU or Leaky ReLU is recommended due to efficient gradient propagation.
- **For binary classification,** Sigmoid is commonly used in the output layer.
- **For Multiclass classification problem,** Softmax is the preferred choice.
- **For deep networks:** Consider using ELU or Leaky ReLU to avoid dead neurons.
- **For regression tasks,** Linear activation is used when predicting continuous values.



Real-World Applications

Neural networks require activation functions as their main component to transform data into complex predictions that achieve accuracy. The applications of activation functions exist throughout real-world scenarios in the following implementations:

1. Image Recognition (ReLU & Softmax in CNNs)

Example: Face Recognition in Smartphones

- In facial recognition systems like Face ID, CNNs analyze facial features using ReLU activation in hidden layers to process pixel values efficiently.
- The final layer uses Softmax activation, which assigns probability scores to different faces to identify the correct person.

2. Natural Language Processing (NLP) (Tanh & Softmax in LSTMs & Transformers)

Example: Chatbots & Virtual Assistants

- Virtual assistants like Alexa, Siri, and Google Assistant use deep learning models with Tanh activation in LSTMs to understand sentence context.
- The last layer of language models uses Softmax activation to predict the most probable next word or response.

Real-World Applications

3. Healthcare – Medical Diagnosis (ReLU & Sigmoid in CNNs & DNNs)

Example: Cancer Detection from Medical Images

- In medical imaging (e.g., detecting tumors from MRI scans), **CNNs with ReLU activation** help extract important image features.
- The final layer uses **Sigmoid activation** to classify an image as either “cancerous” (1) or “non-cancerous” (0), helping doctors with early diagnosis.

4. **Autonomous Vehicles (ReLU & Leaky ReLU in Deep Reinforcement Learning)**

Example: Self-Driving Cars (Tesla Autopilot)

- Self-driving cars process real-time sensor data using deep learning.
- **ReLU activation** in neural networks helps recognize objects like pedestrians, road signs, and vehicles.
- **Leaky ReLU activation** prevents inactive neurons from making split-second driving decisions.

1. **Fraud Detection (Sigmoid in Binary Classification Models)**

- **Example:** Credit Card Fraud Detection
- Banks use AI to detect fraudulent transactions by analyzing patterns in spending behavior.
- A **binary classification neural network** uses **Sigmoid activation** to classify whether a transaction is “fraud” (1) or “legitimate” (0).

Numerical Problems

1. Identity Activation Function
    1. Inputs: $[x_1, x_2, x_3] = [0.5, 0.7, 0.3]$, Weights: $[w_1, w_2, w_3] = [0.2, 0.4, 0.1]$, Bias: $b = 0.1$
    2. Inputs: [0.5, 0.7], Weights: [0.4, 0.6], Bias = 0.2
    3. Inputs: [0.9, 0.4, 0.3], Weights: [0.2, 0.5, 0.1], Bias = 0.0
    4. Inputs: [1.0, 2.0, 3.0], Weights: [0.5, 0.25, 0.1], Bias = 1.0
2. Binary Step Function
3. Bipolar Step Function
    1. Inputs: [0.6,0.4], Weights=[0.5, 0.5], Bias = -0.5
    2. Inputs: [0.2,0.3, 0.1], Weights=[0.6, 0.6, 0.6], Bias = -0.5
    3. Inputs: [1.0,1.0], Weights=[0.7, 0.7], Bias = -1.2
4. Binary Sigmoid
5. Bipolar Sigmoid
6. Hyperbolic Tangent(Tanh)
7. Rectified Linear Unit(ReLU)